Question 1

Explore the Linux directory structure starting from the root directory and verify navigation using commands.

Approach

I started from the root directory '/' and used commands to list available directories. Then I navigated into each directory and verified my location using the 'pwd' command.

Commands

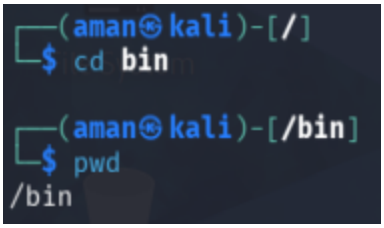
- `cd /` moves to the root directory
- `ls` lists files and directories inside the current directory.
- `cd <directory>` navigates into the specified directory
- `cd ..` is used to move one directory back (parent directory)
- `pwd` confirms the current working directory

Output

STEP 1 : Move to the root directory

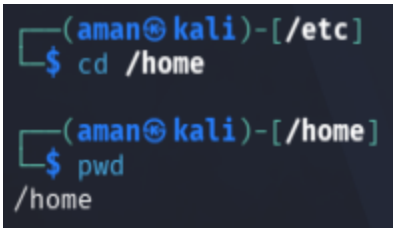
STEP 2 : List root directories

STEP 3 : Navigate to /bin and verify working directory



STEP 4 : Move one directory back (parent directory) and navigate to /etc and verify working directory

STEP 5 : Navigate from /etc to /home and verify working directory



Key Directories Observed

- `/bin` → Essential user binaries
- `/etc` → Configuration files
- `/home` → User directories
- `/var` → Logs and variable data

What I Learned

Linux follows a hierarchical filesystem structure that starts from the root directory (/), allowing organized storage of system and user files.

Question 2

Verify the currently logged-in user using a Linux command.

Approach

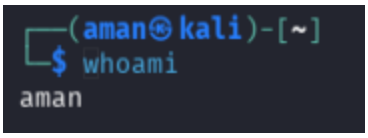
I used a command that displays the username of the active session to confirm which account is currently logged in.

Command

- `whoami` → displays current logged-in username

Output

STEP 1 : check current user



Key Observations

The command directly prints the username associated with the active session

What I learned

The system keeps track of the active user, which is important when performing administrative or restricted actions.

Question 3

Search the system for password wordlists that may be useful for security testing.

Approach

I searched for common password wordlists stored in the system using a file searching command designed for quick lookup.

Commands

- `locate wordlist` → searches system for files containing "wordlist"
- `locate rockyou` → finds common password dictionary files

Output

STEP 1 : search for wordlists

```
┌──(aman㉿kali)-[~]
└─$ locate wordlists
/etc/theHarvester/wordlists
/etc/theHarvester/wordlists/api_endpoints.txt
/etc/theHarvester/wordlists/dns-big.txt
/etc/theHarvester/wordlists/dns-names.txt
/etc/theHarvester/wordlists/dorks.txt
/etc/theHarvester/wordlists/general
/etc/theHarvester/wordlists/names_small.txt
/etc/theHarvester/wordlists/general/common.txt
/usr/bin/wordlists
/usr/lib/python3/dist-packages/theHarvester/wordlists
/usr/share/wordlists
/usr/share/applications/kali-wordlists.desktop
/usr/share/desktop-directories/kali-password-profiling-wordlists.directory
/usr/share/dirb/wordlists
/usr/share/dirb/wordlists/big.txt
/usr/share/dirb/wordlists/catala.txt
/usr/share/dirb/wordlists/common.txt
/usr/share/dirb/wordlists/euskera.txt
/usr/share/dirb/wordlists/extensions_common.txt
/usr/share/dirb/wordlists/indexes.txt
/usr/share/dirb/wordlists/mutations_common.txt
/usr/share/dirb/wordlists/others
/usr/share/dirb/wordlists/small.txt
/usr/share/dirb/wordlists/spanish.txt
/usr/share/dirb/wordlists/stress
/usr/share/dirb/wordlists/vulns
/usr/share/dirb/wordlists/others/best1050.txt
/usr/share/dirb/wordlists/others/best110.txt
/usr/share/dirb/wordlists/others/best15.txt
/usr/share/dirb/wordlists/others/names.txt
/usr/share/dirb/wordlists/stress/alphanum_case.txt
/usr/share/dirb/wordlists/stress/alphanum_case_extra.txt
/usr/share/dirb/wordlists/stress/char.txt
/usr/share/dirb/wordlists/stress/doble_uri_hex.txt
/usr/share/dirb/wordlists/stress/test_ext.txt
/usr/share/dirb/wordlists/stress/unicode.txt
/usr/share/dirb/wordlists/stress/uri_hex.txt
/usr/share/dirb/wordlists/vulns/apache.txt
/usr/share/dirb/wordlists/vulns/axis.txt
/usr/share/dirb/wordlists/vulns/cgis.txt
/usr/share/dirb/wordlists/vulns/coldfusion.txt
/usr/share/dirb/wordlists/vulns/domino.txt
/usr/share/dirb/wordlists/vulns/fatwire.txt
/usr/share/dirb/wordlists/vulns/fatwire_pagenames.txt
/usr/share/dirb/wordlists/vulns/frontpage.txt
/usr/share/dirb/wordlists/vulns/hpsmh.txt
/usr/share/dirb/wordlists/vulns/hyperion.txt
/usr/share/dirb/wordlists/vulns/iis.txt
/usr/share/dirb/wordlists/vulns/iplanet.txt
/usr/share/dirb/wordlists/vulns/jboss.txt
/usr/share/dirb/wordlists/vulns/jersey.txt
```

STEP 2 : search for rockyou wordlist

```
┌──(aman㉿kali)-[~]
└─$ locate rockyou
/usr/share/hashcat/masks/rockyou-1-60.hcmask
/usr/share/hashcat/masks/rockyou-2-1800.hcmask
/usr/share/hashcat/masks/rockyou-3-3600.hcmask
/usr/share/hashcat/masks/rockyou-4-43200.hcmask
/usr/share/hashcat/masks/rockyou-5-86400.hcmask
/usr/share/hashcat/masks/rockyou-6-864000.hcmask
/usr/share/hashcat/masks/rockyou-7-2592000.hcmask
/usr/share/hashcat/rules/rockyou-30000.rule
/usr/share/john/rules/rockyou-30000.rule
/usr/share/wordlists/rockyou.txt.gz
```

Key Observations

Common wordlists are often stored in directories related to security tools.
Example location : `/usr/share/wordlists`

What I learned

Wordlists are used in security testing to perform password attacks such as brute force or dictionary attacks.

Question 4

Create a file using command-line redirection and add additional content to the same file.

Approach

I used output redirection to create a new file and then appended additional text to it without overwriting the original content.

Commands

- `>` → creates new file and writes content
- `>>` → appends content to existing file
- `cat` → displays file content

Output

STEP 1 : Create a new file

```
┌──(aman㉿kali)-[~]
└─$ echo "This is first line" > file.txt
```

STEP 2 : Append text to file

```
┌──(aman㉿kali)-[~]
└─$ echo "This is second line" >> file.txt
```

STEP 3 : View file content

```
┌──(aman㉿kali)-[~]
└─$ cat file.txt
This is first line
This is second line
```

Key Observations

`>` overwrites content, while `>>` adds content to the existing file.

What I learned

Redirection operators allow control over how command output is stored in files.

Question 5

Create a directory and file, then copy the file to the root directory with a new name.

Approach

I created a new directory and file, then copied the file to another location while renaming it.

Commands

- `mkdir` → creates directory
- `touch` → creates empty file
- `cp` → copies file
- `mv` → renames file
- `cd` → change directory

Output

STEP 1 : Create new directory

```
┌──(aman㉿kali)-[~]
└─$ mkdir newfile
```

STEP 2 : Move into directory

```
┌──(aman㉿kali)-[~]
└─$ cd newfile
```

STEP 3 : Create new text file

```
┌──(aman㉿kali)-[~/newfile]
└─$ touch file2.txt
```

STEP 4 : Copy file to root directory

```
┌──(aman㉿kali)-[~/newfile]
└─$ sudo cp file2.txt /root/
[sudo] password for aman:
```

STEP 5 : Rename copied file

```
┌──(aman㉿kali)-[~/newfile]
└─$ sudo mv /root/file2.txt /root/secretfile.txt
```

STEP 6 : Verify file exists

```
┌──(aman㉿kali)-[~]
└─$ ls /root/
ls: cannot open directory '/root/': Permission denied

┌──(aman㉿kali)-[~]
└─$ sudo ls /root
secretfile.txt
```

Key Observations

Administrative privileges may be required to copy files into `/root`

What I learned

Files can be copied and renamed while moving between directories using Linux commands.

SECURITY INSIGHTS FROM CHAPTER 1

Basic Linux commands are essential for cybersecurity tasks such as:

- Identifying active user accounts

- Locating password wordlists used in brute-force testing
- Creating and modifying files during system interaction
- Navigating directories to locate sensitive system data